

Audit + refacto Alice CRM

Alin HERCIU, Mathys PEROLS – EPSI I1 DEV1

1. Les violations MVC : Logique métier et couche de servicee

Le problème : tout était dans les contrôleurs : hash du mot de passe, génération de slug, vérif d'unicité de l'email, nettoyage du SIRET, appel HTTP à Google, renommage des fichiers uploadés. AdminMainController faisait 419 lignes. Rien n'était réutilisable, violant l'une des bases du principe MVC et des couches de services réutilisables.

Ce qui a été fait : création de `src/Service/` avec 4 services : `SlugService`, `UserManager`, `ContactManager` et `MailerService`.

Hasher un mot de passe ou envoyer un mail demande des dépendances externes qu'on ne veut pas trimballer. Avec des services autonomes on peut injecter partout et tester sans bases de données.

2. Code dupliqué

Le problème : chaque repository avait exactement les mêmes lignes, juste réécrites sur son entité (save et remove). Avec aussi beaucoup de code métier recopiés à l'identique.

Ce qui a été fait : Un `AbstractRepository` qui porte `save(object $entity, bool $flush)` et `remove()`, dont tout le monde hérite.

L'autre alternative était de supprimer ces méthodes et d'injecter `EntityManagerInterface` partout mais il fallait réécrire les appels existants. Là, `$repo->save($x, true)` continue de marcher aucun appelant n'a bougé donc nous avons économisé du temps de re-factorisation.

3. Formulaire mal dupliqués

Le problème : `ContactType` et `EditContactType` étaient deux copies du même formulaire, environ 120 lignes identiques. Sauf que les deux avaient divergés sur leur validation : le regex du prénom acceptait les accents à la création mais les refusait à la modification, et la contrainte de longueur sur l'email n'existait que d'un côté.

Ce qui a été fait : Déplacé les contraintes sur l'entité `Contact`, et supprimé `EditContactType` complètement.

4. Sécurité (détecté brute force par IA)

/document n'était protégé par rien. Le `access_control` ne couvrait que `^/admin` et `^/compte`. Tout le `DocumentController` était accessible sans être connecté. Corrigé dans le YAML.

IDOR sur l'édition de contact. N'importe quel utilisateur connecté pouvait modifier le contact d'un autre client en changeant le chiffre dans l'URL. Ajout d'un contrôle d'appartenance qui lève un `AccessDeniedException`.

Énumération de comptes au login. Le message distinguait « vous n'êtes pas enregistré » d'un mot de passe faux, donc on pouvait tester si une adresse existait. Message rendu générique.

5. Problèmes de tests

Le problème : À plusieurs endroits l'entité était utilisée avant d'être testée causant plusieurs erreur 500 ou 404 ou une absence de réponse sur des tokens invalides.

Ce qui a été fait : Les actions reçoivent maintenant l'entité directement en argument (`Contact $contact` au lieu de `$id`), Symfony s'occupe de tout et renvoie un 404 tout seul si elle n'existe pas.

Le problème : `src/Class/Mail.php` était du code mort jamais instancié nulle part. Les clés API passaient en arguments de méthode, le destinataire était codé en dur sur `no-reply@alice-le-blog.fr`, l'ID de template Mailjet était en dur.

Ce qui a été fait : Supprimé le fichier. Créé `MailerService` derrière `MailerServiceInterface` avec un expéditeur injecté depuis les paramètres.

Le problème : Un seul fichier unitaire qui testait des getters/setters générés et 3 fichiers fonctionnels auto-générés par le maker dont presque tout était neutralisé par `markTestIncomplete` et sur des URL qui n'existent pas.

Ce qui a été fait : 3 fichiers de tests unitaires sur les nouveaux services.

Reste à faire (manque de temps)

- Même traitement de validation sur Customer et User
- Sortir le calcul SERP de ContractController
- Réécrire les tests fonctionnels auto-générés -> URL fausses + markTestIncomplete partout